52. Docker Volumes vs Bind Mounts (with MongoDB)

1. The problem: containers forget everything

A container has its own small **writable layer** on top of the image. Anything the app writes, like MongoDB saving your documents into `/data/db`, goes into that layer.

> When you **delete the container** (`docker rm` or `docker compose down`), that writable layer is **deleted with it**. All your MongoDB collections are gone. 😱

```
Container = Image (read-only layers) + thin writable layer
┌──────────────────────────────────┐
│  writable layer  ← /data/db      │  ← deleted on docker rm ✕
├──────────────────────────────────┤
│  mongo:7 image layers (R/O)      │
└──────────────────────────────────┘
```

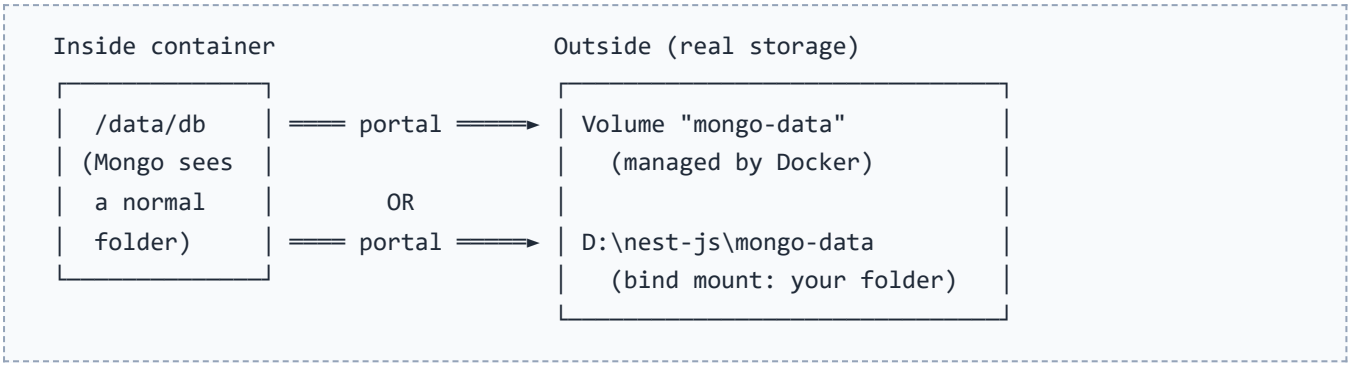
Solution: store the data **outside** the container and plug it in. Docker gives you 2 main ways (+1 special one):

Type	Data lives in...	Who manages it
Volume	A storage area that Docker creates and owns	Docker
Bind mount	A folder **you choose** on your own PC (e.g. `D:\nest-js\mongo-data`)	You
tmpfs	RAM only. It vanishes when the container stops.	Nobody (temporary)

> **Real-life example:**
> **Container** = a hotel room. When you check out, the room is cleaned and everything left there is thrown away.
> **Volume** = the **hotel's locker room**. The hotel gives you locker #12 and decides where it is; you just use it. Your things stay after checkout.
> **Bind mount** = bringing **your own suitcase from home** into the room. It's your bag, you know exactly where it is, and you can open it anytime outside the hotel too.

2. How it works (the "mount" idea)

Mounting means: "make this folder inside the container actually point to a storage place outside it." It's like a **shortcut / portal**.



- MongoDB doesn't know or care. It just writes to `/data/db` as usual.
- The bytes really go to the volume or your folder, so they **survive** container deletion.
- A new container can mount the **same** storage and see the old data.
- Whatever was in the container's `/data/db` before is **hidden** behind the mount (for bind mounts). An empty volume first gets a copy of what the image had there.

3. Docker Volume: in detail

A **volume** is storage that **Docker creates and manages**. You refer to it only by **name**; you don't need to know its path.

Where does it actually live?

- On Linux: `/var/lib/docker/volumes/<name>/_data`
- On Windows/Mac (Docker Desktop): inside Docker's hidden **Linux VM** (WSL 2). It's not a normal folder in Explorer, and that's fine.

```
docker volume inspect mongo-data
# "Mountpoint": "/var/lib/docker/volumes/mongo-data/_data"
```

Volume commands

```
docker volume create mongo-data     # create (optional, run/compose auto-creates)
docker volume ls                    # list all volumes
docker volume inspect mongo-data    # details + location
docker volume rm mongo-data         # delete ⚠ data gone
docker volume prune                 # delete all unused volumes ⚠
```

Named vs anonymous volumes

Named volume	Anonymous volume
`-v mongo-data:/data/db`	`-v /data/db` (no name)
Easy to find and reuse	Gets a random long ID like `a3f9c1…`
✅ Use this	Easy to lose track of

> **Fun fact:** The official `mongo` image declares `VOLUME /data/db /data/configdb` in its Dockerfile. So if you run Mongo **without** `-v`, Docker silently creates **anonymous volumes**. But a fresh `docker run` makes **new** empty ones, so your old data looks "gone". Always give it a **name**.

4. MongoDB with a Named Volume (step by step) ⭐

Step 1: Run Mongo with a volume

```
docker run -d --name mongo \
  -p 27017:27017 \
  -e MONGO_INITDB_ROOT_USERNAME=admin \
  -e MONGO_INITDB_ROOT_PASSWORD=secret \
  -v mongo-data:/data/db \
  mongo:7
```

Read `-v mongo-data:/data/db` as: **volume name** `mongo-data` → plugged into the **container path** `/data/db` (where Mongo stores its files).

Step 2: Add some data

```
docker exec -it mongo mongosh -u admin -p secret

use schooldb
db.students.insertOne({ name: "Gourish", city: "Goa" })
db.students.find()
exit
```

Step 3: Delete the container (the scary part)

```
docker stop mongo
docker rm mongo           # container is GONE
docker volume ls          # but... mongo-data is still here ✅
```

Step 4: Create a brand-new container with the same volume

```
docker run -d --name mongo2 -p 27017:27017 \
  -v mongo-data:/data/db mongo:7

docker exec -it mongo2 mongosh -u admin -p secret
use schooldb
db.students.find()        # → { name: "Gourish", city: "Goa" } 🎉
```

> The container was replaced, but the data survived, because it lived in the **volume** and not in the container. This is also how you **upgrade** Mongo: delete the old container, then run a new image version with the same volume. Check MongoDB's upgrade notes first, because big version jumps need steps.

Step 5: Without a volume (to see the difference)

```
docker run -d --name temp-mongo -p 27018:27017 mongo:7
# insert data … then:
docker rm -f temp-mongo
docker run -d --name temp-mongo -p 27018:27017 mongo:7
# data is gone ❌ (a new anonymous volume was created)
```

5. Bind Mount: in detail

A **bind mount** connects a **folder on your own computer** (one you pick) to a path in the container. You can see and edit those files directly in Explorer or VS Code.

Syntax: it's a bind mount when the left side is a **path**

```
# left side is a PATH (starts with / or ./ or D:\) → bind mount
-v D:/nest-js/mongo-data:/data/db
-v ./mongo-data:/data/db            # (compose, or newer docker CLI)

# left side is a NAME → volume
-v mongo-data:/data/db
```

MongoDB with a bind mount

```
# PowerShell (from your project folder)
mkdir mongo-data
docker run -d --name mongo-bind -p 27017:27017 `
  -v ${PWD}/mongo-data:/data/db `
  mongo:7
```

Now open `mongo-data` in Explorer. You'll see Mongo's real files: `WiredTiger`, `collection-*.wt`, `index-*.wt`, `mongod.lock` ...

> **Don't edit or delete those files by hand** while Mongo is running. They are binary database files, and you'll corrupt the DB. Use `mongosh` or `mongodump` instead.

> **Windows note:** Bind-mounting a database folder from a Windows drive (`C:\`, `D:\`) into a Linux container is **slower**, and it can cause permission or file-locking errors with Mongo. For **database data on Windows, prefer a named volume**. Bind mounts shine for code and config files (next section).
> **Git Bash note:** Git Bash rewrites paths like `/data/db`. Use PowerShell, or prefix the command with `MSYS_NO_PATHCONV=1`.

6. Where bind mounts are really useful (with Mongo)

A) Seed data on the first start: init scripts

The Mongo image runs any `.js` or `.sh` file in `/docker-entrypoint-initdb.d/` **the first time** it starts with an empty database.

```
// mongo-init/seed.js   (on your PC)
db = db.getSiblingDB('schooldb');
db.students.insertMany([
  { name: 'Gourish', city: 'Goa' },
  { name: 'Asha',    city: 'Pune' },
]);
```

```
# volume → data,  bind mount → scripts
docker run -d --name mongo -p 27017:27017 \
  -v mongo-data:/data/db \
  -v ./mongo-init:/docker-entrypoint-initdb.d:ro \
  mongo:7
```

`:ro` = **read-only**. The container can read your scripts but can't change them.

B) Backups to a folder you can see

```
docker run -d --name mongo -p 27017:27017 \
  -v mongo-data:/data/db \
  -v ./backups:/backups \
  mongo:7

docker exec mongo mongodump -u admin -p secret --authenticationDatabase admin --out
/backups/2026-10-19
# → files appear in ./backups on your PC ✅
```

C) NestJS code with live reload (development)

```
-v ./src:/app/src     # edit in VS Code → container sees the change instantly
```

7. Volume vs Bind Mount: side by side

	Volume	Bind mount
Syntax (left side)	A **name**: `mongo-data:/data/db`	A **path**: `./mongo-data:/data/db`
Location	Docker decides (hidden area)	You decide (any folder)
Created by	Docker, automatically	You (the folder must make sense on your PC)
See files in Explorer?	Not easily	Yes ✅
Performance on Windows/Mac	Fast ✅	Slower (crosses Windows ↔ Linux VM)
Permission problems	Rare	More common
Pre-filled from image?	Yes. An empty volume gets the image's files first.	No. Your folder **hides** what the image had there.
Manage with CLI	`docker volume ls/rm/inspect`	Normal file commands
Portable to other machines	Yes (same name works anywhere)	Depends on that path existing
Best for	**Database data** (Mongo, Postgres, Redis)	**Source code, config files, init scripts, backups**

> **Easy rule:**
> 🛍️ **Data that only the app touches** (DB files) → **Volume**
> 📝 **Files you want to see or edit yourself** (code, configs, seeds) → **Bind mount**

8. tmpfs (the third type, briefly)

```
docker run -d --tmpfs /tmp mongo:7
```

It's stored only in **RAM** and lost when the container stops. It's used for temporary or secret data that should never touch disk. You can't use it for DB data you want to keep.

9. `-v` vs `--mount` syntax

They do the same thing. `--mount` is longer but clearer:

```
# Volume
-v mongo-data:/data/db
--mount type=volume,source=mongo-data,target=/data/db

# Bind mount (read-only)
-v ./mongo-init:/docker-entrypoint-initdb.d:ro
--mount type=bind,source=./mongo-init,target=/docker-entrypoint-initdb.d,readonly
```

> One difference: if a bind-mount folder doesn't exist, `-v` **creates it** (empty), while `--mount` **gives an error**. That error helps you catch typos.

10. Docker Compose: Mongo + NestJS (both types together)

```
services:
  mongo:
    image: mongo:7
    container_name: mongo
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: secret
    volumes:
      - mongo-data:/data/db                                  # 🛍️ VOLUME  (DB files)
      - ./mongo-init:/docker-entrypoint-initdb.d:ro          # 📝 BIND    (seed scripts)
      - ./backups:/backups                                   # 📝 BIND    (backups)

  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      MONGO_URI: mongodb://admin:secret@mongo:27017/schooldb?authSource=admin
    volumes:
      - ./src:/app/src                                       # 📝 BIND    (live code)
    command: npm run start:dev
    depends_on:
      - mongo

volumes:          # ← named volumes must be declared here
  mongo-data:
```

```
docker compose up -d       # start
docker compose down        # remove containers → mongo-data volume KEPT ✅
docker compose down -v     # remove containers AND volumes → data DELETED ❌
```

> Compose adds the project name as a prefix, so the real volume name becomes `my-app_mongo-data`. Check with `docker volume ls`.

Connect NestJS (Mongoose) to it

```
// app.module.ts
MongooseModule.forRoot(process.env.MONGO_URI),

// running Nest on your laptop (not in Docker)?
//   mongodb://admin:secret@localhost:27017/schooldb?authSource=admin
// running Nest inside compose?
//   mongodb://admin:secret@mongo:27017/schooldb?authSource=admin   (service name)
```

11. Common problems and fixes

Problem	Why	Fix
Data gone after restart	No named volume was used, or you ran `down -v` / `volume prune`	Always use `-v mongo-data:/data/db`
Changed `MONGO_INITDB_ROOT_PASSWORD` but the old password still works	Init variables and init scripts run **only when `/data/db` is empty** (first start)	Change the user inside mongosh, or delete the volume to start fresh
`seed.js` didn't run	Same reason: the volume already has data	`docker compose down -v`, then `up` again (⚠ wipes data)
Mongo crashes with a bind mount on Windows	File-system or locking issues across Windows ↔ Linux	Use a named volume for `/data/db`
Bind folder is empty inside the container	Wrong path (typo), so Docker created a new empty folder	Check the path, or use `--mount` to get an error
Container files "disappeared" after a bind mount	The bind mount **hides** what was at that path in the image	Mount to a different path, or copy the files into your folder
Can't remove a volume	A container (even a stopped one) still uses it	`docker rm` the container first

12. Backup and restore a volume

Mongo way (recommended)

```
# backup
docker exec mongo mongodump -u admin -p secret --authenticationDatabase admin --
archive=/backups/db.archive
# restore
docker exec mongo mongorestore -u admin -p secret --authenticationDatabase admin --
archive=/backups/db.archive
```

Generic way (any volume): copy it to a tar file using a helper container

```
# stop mongo first so the files are consistent
docker stop mongo
docker run --rm -v mongo-data:/data -v ${PWD}:/backup alpine \
  tar czf /backup/mongo-data.tar.gz -C /data .
docker start mongo
```

Here a throwaway `alpine` container mounts the **volume** and a **bind mount** of your current folder at the same time, then zips one into the other. This shows nicely how both types work together.

13. Quick summary

Question	Answer
Why do we need them?	Container data is deleted with the container. Mounts keep it outside.
What is a volume?	Docker-managed storage, referred to by **name**
What is a bind mount?	A folder **on your PC**, referred to by **path**
How do they work?	A container path (like `/data/db`) becomes a portal to storage outside the container
Mongo data folder	`/data/db`
Mongo seed folder	`/docker-entrypoint-initdb.d` (runs only on the first start)
Use a volume for	Database files ✅
Use a bind mount for	Code, configs, seed scripts, backups ✅
Danger commands	`docker compose down -v`, `docker volume rm`, `docker volume prune`